



POLITÉCNICA

UNIVERSIDAD POLITECNICA DE MADRID

Higher Technical School of Engineering and Telecommunication Systems
Department of Telematics and Electronic Engineering

OPERATING SYSTEMS

Practice 3

Lightweight Virtualization with Docker and Kubernetes

Step-by-step record of the laboratory procedure: containers, images, resource limits, Docker Compose and orchestration on a minikube cluster.

Student	Guillermo Aladro Abad
Degree	Telecommunication Systems Engineering
Subject	Operating Systems - academic year 2025/2026
Environment	Debian GNU/Linux virtual machine, user estudiante
Repository	github.com/GuillermoAladro/UPMPProjects - Operativos/P3-Virtualizacion-Docker-K8s

Contents

1. Introduction	3
2. Objectives	3
3. Working environment	3
4. Part I - Containers with Docker	3
4.1 Installing containerd and Docker	3
4.2 Container lifecycle	4
4.3 Images	5
4.4 Assigning external resources	5
4.5 Limiting resource usage	5
4.6 Use case: WordPress with two containers	6
4.7 The same application with Docker Compose	6
5. Part II - Orchestration with Kubernetes	6
5.1 Installing minikube	6
5.2 Fundamental concepts	7
5.3 Declaring pods	7
5.4 Declaring network services	7
5.5 Persistent storage	8
5.6 Declarative configuration of applications	8
6. Conclusions	9
7. Repository contents	9
Appendix A. Configuration files	11
A.1 docker/wordpress.yaml	11
A.2 kubernetes/nginx.yaml	11
A.3 kubernetes/ubuntu.yaml	13
A.4 kubernetes/kustomization.yaml	13

1. Introduction

Lightweight virtualization runs an application and its dependencies in an isolated user space while sharing the kernel of the host operating system. Unlike a virtual machine, there is no hypervisor and no second kernel: isolation is provided by kernel features, namely namespaces for the filesystem, process table, users and network, and control groups for limiting CPU, memory and I/O. The result starts in milliseconds and consumes a fraction of the resources, at the cost of portability across different operating systems.

This document records, step by step, the procedure followed in Practice 3 of the Operating Systems course. The first half covers container management with Docker on a single machine; the second half covers orchestration with Kubernetes on a single-node minikube cluster. Every command is given as it is executed in the Debian virtual machine of the practice, together with an explanation of what the operating system does underneath and why the step matters.

The configuration files quoted throughout the report are kept in the repository, so the deployment can be reproduced without transcribing them from these pages.

2. Objectives

- Understand the foundations of lightweight virtualization with containers.
- Deploy applications with containers in a local environment using Docker.
- Deploy applications on a Kubernetes cluster and understand the declarative model.

The practice is assessed through a laboratory test rather than a submitted report; this document is therefore a technical record of the work, written to be usable both as a portfolio piece and as preparation material.

3. Working environment

Host system	Debian GNU/Linux virtual machine supplied with the practice
User	estudiante (member of the docker group after installation)
Resources	at least 2 CPUs and 4 GiB of RAM, required by minikube
Runtime	containerd, driven by Docker Engine and the compose plugin
Orchestrator	minikube with kubectl, running on top of the same Docker installation

4. Part I - Containers with Docker

4.1 Installing containerd and Docker

The installation adds the official Docker repository to APT and installs the engine, the CLI, the containerd runtime and the buildx and compose plugins. It is run as superuser; the rest of the practice is carried out as the unprivileged user.

```
apt update
apt upgrade
apt install apt-transport-https ca-certificates curl gnupg

install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc

tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

apt update
apt install docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin

adduser estudiante docker
```

Two details are worth noting. The keyrings directory does not exist on a clean Debian installation, so it has to be created before the key is downloaded into it. And the group membership granted in the last line only takes effect in a new session: the Docker socket is owned by root and group docker, which is precisely what allows the student to use the engine without sudo.

```
id                # estudiante must now appear in the docker group
docker run hello-world
docker images
```

4.2 Container lifecycle

Three containers are started in different ways, which is enough to show the difference between a detached container, a short-lived one and a foreground one that occupies the terminal until it ends.

```
docker run --detach eclipse-mosquitto    # detached, long-running broker
docker run -d ubuntu sleep 1000         # detached, ends after 1000 s
docker run --name servidorweb nginx      # foreground: a second session is needed
```

From another session the containers are inspected and controlled. The image only carries what the application needs, so the usual diagnostic tools normally have to be installed inside the container before they can be used.

```
docker ps                # running containers
docker logs <id|name>    # standard output of a container
docker exec -it <id|name> bash # a shell inside the container

# inside the container
id; df; mount; cat /etc/hosts; ps ax; netstat -an; ifconfig
apt update && apt install procs net-tools inetutils-ping curl
```

The output of those commands is the core observation of the section: the container sees its own filesystem, its own process table numbered from PID 1, its own network interfaces and its own hosts file, while the kernel executing all of it is the one of the virtual machine. Isolation is therefore a matter of what the kernel shows to each group of processes, not of emulating a second machine.

```

docker cp servidorweb:/usr/share/nginx/html/index.html .
docker cp fichero.txt servidorweb:/tmp

docker stop <id|name>      # SIGTERM, ordered shutdown
docker kill <id|name>      # SIGKILL, abrupt
docker start <id|name>
docker restart <id|name>
docker rm <id|name>        # definitive removal

docker stats               # live CPU and memory usage per container
docker network ls
docker network inspect <id|name>

```

The distinction between stop and kill is the same one studied with processes in the previous practice: the first delivers SIGTERM and gives the application the chance to close cleanly, the second delivers SIGKILL and the process cannot react to it.

4.3 Images

Applications are distributed as images in the OCI standard format, pulled from public registries such as Docker Hub. An image is a stack of read-only layers; a container adds a writable layer on top of it, which is discarded when the container is removed.

```

docker search nginx
docker pull nginx
docker images
docker history nginx      # the layers the image is made of
docker rmi nginx

```

4.4 Assigning external resources

A container is useful only if it can reach the outside world: durable storage, network ports and configuration. The three mechanisms are applied in turn.

```

# persistent storage: a host directory mounted inside the container
docker run --volume ~/contenedores/c1:/datos -d ubuntu sleep 1000

# network: a host TCP port redirected into the container
docker run --publish 12345:80 -d nginx
curl http://localhost:12345

# configuration: an environment variable injected at start-up
docker run --name prueba --env VARIABLE=valor -d ubuntu sleep 1000

```

The volume is what makes the data outlive the container: files written under /datos are still in the host directory after the container is deleted, whereas anything written elsewhere disappears with the writable layer.

4.5 Limiting resource usage

Control groups are what keep a container from taking the whole machine. A limited container is created and then deliberately stressed.

```

docker run --cpus 1.5 --memory 128m --name limitado -d ubuntu sleep 1000

docker exec -it limitado bash
apt update && apt install stress-ng
stress-ng --vm 4 --vm-bytes 1G --timeout 60s

```

Observing the load with `top` on the host and `docker stats` side by side shows the cgroup at work: however many workers `stress-ng` starts, the container does not exceed the CPU quota it was given, and its memory stays at the ceiling because workers that try to go beyond it are terminated by the out-of-memory killer instead of pushing the host into swap.

4.6 Use case: WordPress with two containers

The section culminates in a real two-tier application: a MariaDB container holding the database and a WordPress container serving the site, each one parameterized through environment variables and given a host directory for its persistent data.

```
docker run -d --name servidormdb \
  -e MYSQL_ROOT_PASSWORD=sesamo \
  -e MYSQL_DATABASE=bdwp \
  -e MYSQL_USER=usuwp \
  -e MYSQL_PASSWORD=contrawp \
  -v ~/contenedores/servidormdb:/var/lib/mysql \
  mariadb

docker run -d --name servidorwp \
  -e WORDPRESS_DB_HOST=servidormdb:3306 \
  -e WORDPRESS_DB_NAME=bdwp \
  -e WORDPRESS_DB_USER=usuwp \
  -e WORDPRESS_DB_PASSWORD=contrawp \
  -v ~/contenedores/servidorwp:/var/www/html/wp-content \
  -p 80:80 --link servidormdb:3306 \
  wordpress

docker logs servidorwp
```

The site is then reachable on port 80 of the virtual machine. A bind error on that port means it is already taken by an earlier activity, in which case the previous container is removed or a different port is published. To start the activity over, both containers and their persistent directories have to be deleted, otherwise WordPress finds the previous database and skips the installation screen.

4.7 The same application with Docker Compose

Compose replaces the two commands above with a single declarative file. The stack is given different names and port 9001 so that it does not collide with the manual deployment, and named volumes managed by Docker replace the host directories.

```
docker compose -f wordpress.yaml up -d
docker compose -f wordpress.yaml ps
docker compose -f wordpress.yaml logs
docker compose -f wordpress.yaml stop | start | restart | rm
```

Compose creates a dedicated network for the project, which is why the WordPress container reaches the database simply as `serv_mariadb`, without the legacy `--link` option. The complete file is reproduced in Appendix A.

5. Part II - Orchestration with Kubernetes

5.1 Installing minikube

`minikube` implements a complete Kubernetes cluster on a single node, on top of the Docker and `containerd` already installed, and is the reason the virtual machine needs two processors and four

gigabytes of memory.

```
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
minikube start
sudo apt install kubectl

kubectl get nodes -o wide
kubectl get namespaces
kubectl get all
```

Access to the Kubernetes API is always authenticated and encrypted; minikube writes the necessary credentials to `~/.kube/config`, which is what lets kubectl administer the cluster straight away.

5.2 Fundamental concepts

Everything the cluster manages is represented as an object stored in the etcd database and exposed through the API server of the control plane. Management is declarative: the administrator declares the desired state, and the controllers of the control plane plan and carry out whatever is needed to reach it, instructing each worker node to create, modify or delete containers, virtual networks and storage. On every worker node a kubelet process manages the containers of that node and a kube-proxy process manages the virtual networking with the rest of the cluster. In minikube all of this runs at reduced scale on the local machine.

5.3 Declaring pods

A pod is the smallest deployable unit. It is declared in YAML and applied with kubectl; the same file re-applied after a change is how objects are modified.

```
kubectl apply -f nginx.yaml

kubectl get pods
kubectl get pod/nginx -o yaml      # the object as the cluster stores it
kubectl describe pod/nginx        # events: scheduling, image pull, start
kubectl exec -it pod/nginx -- bash

kubectl port-forward pod/nginx 8080:80
curl -D - http://localhost:8080

# port 80 is privileged, so this form needs sudo and an explicit kubeconfig
sudo kubectl --kubeconfig=$HOME/.kube/config port-forward pod/nginx 80:80
```

Changing the image tag from `nginx:1.25` to `nginx:1.29` in the file and applying it again illustrates the declarative model: nothing describes how to upgrade, only the desired end state, and the control plane destroys the old pod and creates a new one with the new image. Only some properties of an object can be modified in place; for the rest the object has to be destroyed and created again.

A second pod running Ubuntu is declared as well, to be used as a client from inside the cluster. It is given a long sleep as its command, because a container whose process finishes immediately would leave the pod restarting in a loop.

5.4 Declaring network services

Port forwarding is only a testing tool, and pod addresses are neither stable nor reachable from outside the cluster. The practice makes this explicit by trying the wrong ways first, each of which fails for its own reason.

Attempt	Result
curl http://<pod IP>:80 from the host	no connection: the pod address is not routable outside the cluster
curl http://localhost:80 from the Ubuntu pod	no connection: localhost is that pod's own network namespace
curl http://nginx:80 from the Ubuntu pod	the name cannot be resolved: a pod name is not a DNS record
curl http://<pod IP>:80 from the Ubuntu pod	it connects, but the address changes whenever the pod is recreated

The solution is a Service object. It gives the application a stable name and virtual address inside the cluster, and it binds to the pods that implement it through label selectors rather than by name, so the set of pods behind the name can change freely.

```
kubectl apply -f nginx.yaml      # pod with labels + service
kubectl get services

kubectl exec -it pod/base -- bash
curl http://servidorweb:80      # now it resolves and connects
```

The service created here is of type ClusterIP, which is reachable from inside the cluster only. Exposing an application to the outside requires a service of type NodePort or LoadBalancer, or an Ingress. The resources are then removed, either one by one or through the files that declared them.

```
kubectl delete pod/nginx
kubectl delete pod/base
kubectl delete service/servidorweb
# or, equivalently
kubectl delete -f nginx.yaml -f ubuntu.yaml
```

5.5 Persistent storage

As with plain containers, a pod keeps nothing by itself. Persistent storage is requested through a PersistentVolumeClaim: the pod asks for a capacity and an access mode and never names a disk, and a CSI provisioner satisfies the claim with whatever the cluster has available.

```
kubectl apply -f ubuntu.yaml
kubectl get pvc
kubectl get pv
kubectl exec -it base -- bash
df                          # /datos is backed by the claim
```

In minikube the claim is satisfied by the hostpath provisioner, which stores the volume on the node filesystem under `/var/lib/docker/volumes/minikube/_data/hostpath-provisioner`. In a production cluster the same declaration would be served by shared storage such as NFS, Ceph or EBS, without the pod definition changing at all.

5.6 Declarative configuration of applications

The final activity deploys the WordPress and MySQL example from the Kubernetes documentation. The upstream manifests are downloaded, the service type is changed from LoadBalancer to NodePort because minikube has no cloud load balancer behind it, and a kustomization generates the database password as a Secret instead of leaving it written in the manifests.

```
curl -LO https://k8s.io/examples/application/wordpress/mysql-deployment.yaml
curl -LO https://k8s.io/examples/application/wordpress/wordpress-deployment.yaml

sed -i '' 's/LoadBalancer/NodePort/' wordpress-deployment.yaml

kubectl apply -k ./
minikube service wordpress --url
```

Unlike the bare pods of the previous sections, these manifests declare Deployments. A Deployment is a persistent statement of intent: the controller creates the pods needed to implement the application and, if one of them disappears because its node restarted or the process terminated abnormally, creates another one on some node of the cluster to restore the declared state.

6. Conclusions

Isolation is a kernel service. A container is a set of processes with their own namespaces and a cgroup that bounds their resources. There is no second kernel and no hypervisor, which is why start-up is immediate and the overhead is small, and also why portability between different operating systems is limited compared with a full virtual machine.

Nothing persists unless it is declared. The writable layer of a container dies with it. Durable state has to be attached explicitly, as a host directory or a named volume in Docker and as a PersistentVolumeClaim in Kubernetes, and the claim is the more expressive of the two because it separates what the application needs from what the infrastructure provides.

Configuration is injected, never baked in. The same public mariadb and wordpress images become this particular deployment through environment variables, published ports and mounted volumes. Kubernetes generalizes this with ConfigMaps and Secrets, which is what allows one image to serve every environment.

Imperative versus declarative is the real divide. The Docker half of the practice issues orders; the Kubernetes half declares an end state and delegates the path to it. That is what makes self-healing possible: a Deployment is not a command that has already been executed but a condition the cluster keeps enforcing.

Naming is what makes a cluster usable. Pod addresses are unstable and private, so services and label selectors provide the indirection layer that lets the set of running pods change without anything that depends on them being rewritten.

7. Repository contents

The files described in this report are kept under Operativos/P3-Virtualizacion-Docker-K8s in the UPMPProjects repository:

Path	Content
docker/install-docker.sh	installation of containerd and Docker on Debian

docker/runbook.md	command walkthrough of the Docker half
docker/wordpress.yaml	WordPress and MariaDB stack for Docker Compose
kubernetes/nginx.yaml	nginx pod and the ClusterIP service exposing it
kubernetes/ubuntu.yaml	auxiliary pod with a PersistentVolumeClaim
kubernetes/kustomization.yaml	WordPress on Kubernetes with a generated Secret
kubernetes/runbook.md	command walkthrough of the Kubernetes half

Guillermo Aladro Abad

Telecommunication Systems Engineering - ETSIST, UPM

Appendix A. Configuration files

A.1 docker/wordpress.yaml

```
# Question 13 - The same WordPress + MariaDB stack as Questions 11 and 12,
# declared for Docker Compose instead of two docker run commands.
# Names and the published port differ from the manual deployment so both
# can coexist on the same host.
#
# docker compose -f wordpress.yaml up -d
# docker compose -f wordpress.yaml ps | logs | stop | rm
#
# The site is then served on http://localhost:9001
services:
  serv_mariadb:
    image: mariadb
    container_name: serv_mariadb
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: supercontra
      MYSQL_DATABASE: bdatos_wp
      MYSQL_USER: usu_wp
      MYSQL_PASSWORD: contra_wp
    volumes:
      - datos_mariadb:/var/lib/mysql

  serv_wp:
    image: wordpress
    container_name: serv_wp
    restart: always
    depends_on:
      - serv_mariadb
    environment:
      WORDPRESS_DB_HOST: serv_mariadb
      WORDPRESS_DB_NAME: bdatos_wp
      WORDPRESS_DB_USER: usu_wp
      WORDPRESS_DB_PASSWORD: contra_wp
    ports:
      - "9001:80"
    volumes:
      - datos_wp:/var/www/html/wp-content

# Named volumes managed by Docker: the databases and the WordPress content
# survive the removal of the containers.
volumes:
  datos_mariadb:
  datos_wp:
```

A.2 kubernetes/nginx.yaml

```
# Questions 18, 20 and 23 - nginx pod plus the service that exposes it.
#
# The pod alone is reachable only through `kubectl port-forward`; its IP is
# neither stable nor routable from outside the cluster. The Service gives it a
# stable name (servidorweb) inside the cluster, and finds its backing pods
# through the label selector rather than by name.
#
#   kubectl apply -f nginx.yaml
#   kubectl get pods -o wide
#   kubectl get services
#   kubectl exec -it pod/base -- bash   # then: curl http://servidorweb:80
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app.kubernetes.io/name: ServidorP3
spec:
  containers:
    - name: nginx
      # Question 20: changing this tag and re-applying the file makes the control
      # plane destroy the pod and create a new one from the new image.
      image: nginx:1.25
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: servidorweb
spec:
  # ClusterIP is reachable from inside the cluster only. Exposing the service
  # outside minikube would need NodePort, LoadBalancer or an Ingress.
  type: ClusterIP
  selector:
    app.kubernetes.io/name: ServidorP3
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

A.3 kubernetes/ubuntu.yaml

```
# Questions 21 and 25 - auxiliary Ubuntu pod with persistent storage.
#
# The pod is used as a client from inside the cluster (Question 22), and its
# /datos directory is backed by a PersistentVolumeClaim, so the data survives
# the pod being deleted and recreated. In minikube the claim is satisfied by
# the hostpath provisioner, which stores the volume on the node's filesystem
# under /var/lib/docker/volumes/minikube/_data/hostpath-provisioner.
#
# kubectl apply -f ubuntu.yaml
# kubectl get pvc
# kubectl get pv
# kubectl exec -it base -- bash      # then: df
apiVersion: v1
kind: Pod
metadata:
  name: base
spec:
  containers:
  - name: ubuntu
    image: ubuntu:latest
    # Without a long-running command the container would exit immediately and
    # the pod would restart in a loop.
    command: ["/bin/sleep", "30d"]
    volumeMounts:
    - name: datos-ubuntu
      mountPath: /datos
  volumes:
  - name: datos-ubuntu
    persistentVolumeClaim:
      claimName: datos-ubuntu
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: datos-ubuntu
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
```

A.4 kubernetes/kustomization.yaml

```
# Question 26 - declarative WordPress deployment on minikube.
#
# The two upstream manifests are downloaded first, and the WordPress service is
# switched from LoadBalancer to NodePort because minikube has no cloud load
# balancer behind it:
#
#   curl -LO https://k8s.io/examples/application/wordpress/mysql-deployment.yaml
#   curl -LO https://k8s.io/examples/application/wordpress/wordpress-deployment.yaml
#   sed -i '' 's/LoadBalancer/NodePort/' wordpress-deployment.yaml
#
# Then this kustomization generates the database password as a Secret and
# applies everything together:
#
#   kubectl apply -k ./
#   minikube service wordpress --url
#
# Unlike the bare pods above, these manifests declare Deployments, so the
# control plane recreates a pod that dies or whose node restarts.
secretGenerator:
- name: mysql-pass
  literals:
  - password=YOUR_PASSWORD
resources:
- mysql-deployment.yaml
- wordpress-deployment.yaml
```